Full Tree – 0 or 2 Children

Complete Tree – Perfect until h-1 (NO GAPS)

Perfect Tree – 2^height – 1

Removing: ***Set to largest item in left subtree***

Traversals:

"InOrder!" →Left,Root,Right

PreOrder → Root,Left,Right

PostOrder →Left,Right,Root

Min-heap *(Must be complete!!)*

think weights of the values; L → R

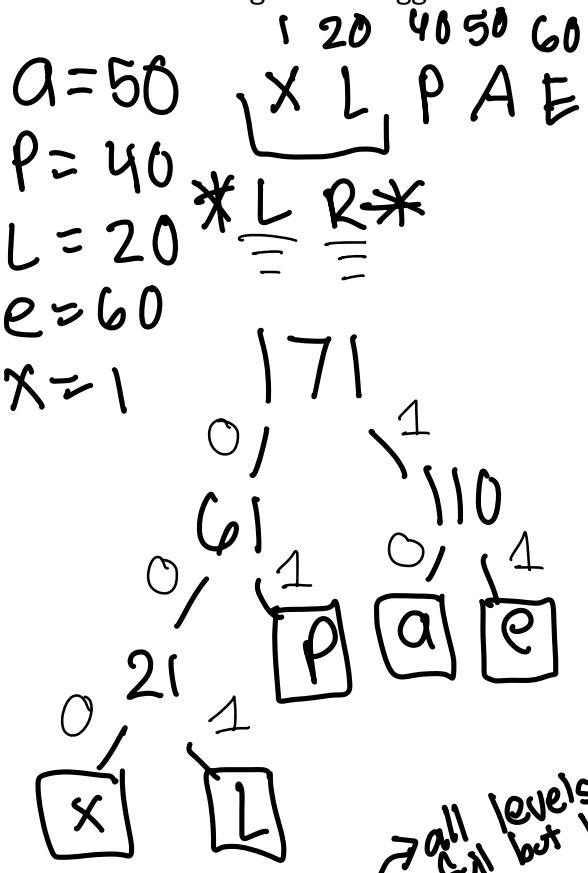
Inserting: Swap with parent if bigger

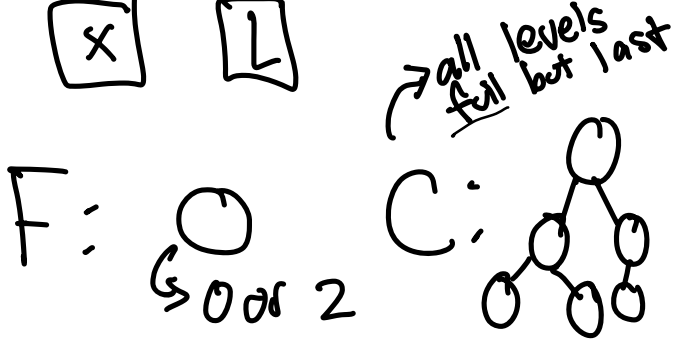
Removing: Root becomes highest right; sift weight

Priority Queue Low → High

Queue: FIFO, Stack: LIFO

Tree Vocab: child, Edges, Parent Siblings, Ancestor Descendant. Height = 1 + biggest subtree





Open Addressing - look for open; Chaining - LinkedList

Huffman Trees: 0's L ; 1's R

Think **base cases and **recursion** when doing trees**

```
Public int sizeTree(Node<E> root){
        if(root == null){  // Base case empty tree
            return 0;
        } // 1 is the counting of the root itself
        return sizeTree(root.left) + sizeTree(root.right) + 1
        }
Public static<E> boolean isFull(Node<E> root){
        if(root == null) {
            return true;
        }
        } else if (root.left == null &&  root.right == null) {
            Return true;
        } else if(root.left != null && root.right == null){
            Return false;
        } else if(root.left == null && root.right != null){
            Return false;
        } else {
            Return isFull(root.left) && isFull(root.right)
        }
```

```
Public static Map<Character,Integer> count(String word){
        Map<Character,Integer> count = new HashMap<>();
        for(int I = 0; I < word.length();i++){
        if(count.containsKey(word.charAt(i) == false){
            count.put(word.charAt(i), 1);
        } else { // Count returns the value not the key
Count.put(word.charAt(i), count.get(word.charAt(i))+ 1);
```

HashSet<String> cars = new...
cars.add( ), cars.remove( )
cars.contains( )
* No duplicate elements *

```
public static Map<Integer,List<Integer>> splitNumbersByFactors(int [] arr){
    Map<Integer,List<Integer>> factors = new HashMap<>();
    factors.put(2, new ArrayList<>());
    factors.put(3, new ArrayList<>());
    factors.put(5, new ArrayList<>());
    for(int i = 0; i < arr.length;i++){
        if(arr[i] % 2 == 0){
            factors.get(2).add(arr[i]);
        }
        if(arr[i] % 3 == 0){
            factors.get(3).add(arr[i]);
        }
        if(arr[i] % 5 == 0){
            factors.get(5).add(arr[i]);
        }
    }
    return factors;
}
```

Key = ID , Value = Value(s)